

# Slimes

protocolos, parsers e funções puras - na prática

Zoey de Souza Pessanha - @zoedsoupe

SCTI - UENF - 2026

# Seu código vai jogar contra o de todo mundo aqui.

---

numa rede que perde mensagens. de propósito.

# O que hoje **é** (e o que **não é**)

- não é aula de JavaScript - JS é só a ferramenta que todo computador do lab já tem
- não é competição de algoritmo - o torneio é a desculpa, não o objetivo
- é construir um programa que conversa com um servidor pela rede
- é aprender, na prática, como programas trocam mensagens sem se perder
- é 3 horas e meia, sozinho ou em dupla (vocês escolhem), com intervalo e torneio no fim

# Como a gente vai aprender

1. **Prever**: antes de ver qualquer código, vocês chutam como funciona
2. **Rodar e investigar**: abrem o código pronto e comparam com o palpite
3. **Modificar e construir**: só então escrevem o código de vocês

sempre nessa ordem: primeiro entender, depois escrever.

# O dia

| horário   | o que acontece                                                      |
|-----------|---------------------------------------------------------------------|
| 0:10-0:25 | <b>prever:</b> o que o servidor manda a cada segundo?               |
| 0:25-1:00 | <b>rodar + investigar:</b> kit aberto, testes vermelhos, só leitura |
| 1:00-1:35 | <b>construir</b> E1 e E2: testes verdes no simulador                |
| 1:35-1:45 | intervalo                                                           |
| 1:45-2:30 | <b>construir</b> E3: sua estratégia contra os bots                  |
| 2:30-2:50 | servidor real, fase <b>cooperativa</b>                              |
| 2:50-3:00 | as regras mudam: a sala inteira conserta junto                      |
| 3:00-3:20 | rodadas do <b>torneio</b>                                           |
| 3:20-3:30 | demo final + fechamento                                             |

# PARTE 1

---

## O problema

antes de qualquer código: o que estamos construindo?

# Pergunta pra sala

20 computadores querem jogar **o mesmo jogo**, ao mesmo tempo, cada um na sua máquina.

**Como eles combinam as regras e sabem o que aconteceu?**

# O juiz: o servidor

- o servidor é o **único** que conhece o jogo inteiro
- cada jogador roda um programa: o **cliente** - é ele que vocês vão escrever
- o cliente **pergunta** e **obedece**: quem decide se a jogada vale é o servidor
- servidor e cliente conversam trocando **mensagens de texto** pela rede

hoje, o jogo é o **Slimes**. mas esse desenho - um servidor, muitos clientes - é o mesmo de um banco, de um chat, de uma loja online.



# As regras do Slimes

cinco regras. é tudo que vocês precisam saber.

# Regra 1: o mundo é uma grade

- o mundo é uma grade de **60 por 40** casas
- cada pessoa (ou dupla) controla uma **colônia** de slime: um conjunto de casas na grade
- sua colônia começa pequena, num canto do mapa

## Regra 2: o tempo anda em ticks

- o jogo avança **um tick por segundo** - como os segundos de um relógio
- a cada tick, cada colônia faz **uma ação**
- o servidor junta as ações de todo mundo, aplica as regras e conta o resultado

# Regra 3: quatro ações possíveis

- `expand` - crescer pra uma casa vizinha vazia
- `attack` - tomar uma casa vizinha do inimigo
- `fortify` - proteger uma casa sua: ela passa a resistir a ataque
- `pass` - não fazer nada nesse tick

uma ação por tick. escolher bem **qual** ação é o coração do jogo - e é o exercício 3.

# Regra 4: você não vê o jogo inteiro

- seu cliente só enxerga a **vizinhança 7×7** ao redor das suas casas
- o resto do mapa é escuridão: você joga com informação parcial
- é como dirigir à noite: você só vê o que o farol ilumina

sistemas de verdade são assim: nenhum programa vê o mundo inteiro, só a parte que chega até ele.

# Regra 5: sem casas, fim de jogo

- se a sua colônia perder todas as casas, ela **morre**
- mas ninguém sai da sala: eliminado vira **espectador** e assiste pelo projetor

dois modos de jogo: **cooperativo** (ataques desligados, todo mundo cresce no seu canto) e **torneio** (vale tudo).

# O servidor é a autoridade.

---

ele nunca confia que o seu cliente é educado.  
o seu cliente nunca confia que a rede é confiável.

# PARTE 2



## A conversa

como cliente e servidor trocam mensagens



# Pergunta pra sala

Seu programa e o servidor estão em máquinas diferentes.

**Como um fala com o outro?**

# Parte 1: a linha telefônica (WebSocket)

- o cliente **liga** pro servidor e a linha **fica aberta** o jogo inteiro
- os dois lados podem falar a qualquer momento, sem desligar e ligar de novo
- o nome dessa "linha" é **WebSocket** - o navegador já sabe fazer isso, vocês não escrevem essa parte

diferente de uma carta (manda, espera, manda outra): é uma ligação que não desliga.

## Parte 2: o idioma combinado (protocolo)

- a linha estar aberta não basta: os dois lados precisam falar **o mesmo idioma**
- esse idioma combinado é o **protocolo**: uma lista de mensagens, cada uma com um formato exato
- o nosso protocolo é **texto puro**: uma mensagem por linha, legível a olho nu, zero JSON

por que texto e não JSON? porque vocês vão **escrever o tradutor** com as próprias mãos - e vão conseguir ler o tráfego inteiro nas ferramentas do navegador.

# Como é uma conversa de verdade

uma mensagem por linha. andar linha por linha no clique.

```
HELLO v1 aurora-k3f9-1 colony aurora
WELCOME srv-0 3 aurora 96CDFB 60 40 1000 3 4,34

ACT aurora-k3f9-17 expand 12 7
ACK aurora-k3f9-17 97

OBS srv-97 97 alive 97 9,5,plain,0,0;10,5,plain,3,1
SCORE srv-97 97 3,aurora,21,alive;5,nova,14,dead
NACK aurora-k3f9-18 too_late tick 96 resolvido
```

# Lendo a primeira mensagem

```
HELLO v1 aurora-k3f9-1 colony aurora
```

- `HELLO` : o **tipo** da mensagem - toda linha começa assim, com um nome em maiúsculas
- `v1` : a **versão** do protocolo que o cliente fala (guarde esse `v1`, ele volta às 2:50)
- `aurora-k3f9-1` : uma **etiqueta única** dessa mensagem - a gente já chega nela
- `colony aurora` : o **conteúdo** - aqui, o nome da colônia

toda mensagem é assim: **tipo** primeiro, depois os **campos**, separados por espaço.

# Anatomia de uma OBS

a mensagem que conta o que você enxerga, a cada tick

```
OBS srv-97 97 alive 97 9,5,plain,0,0;10,5,plain,3,1
```

- `srv-97`: a etiqueta do servidor - sempre `srv-` seguido do tick
- `97`: o tick que essa visão descreve
- `alive`: sua colônia está viva; depois de eliminado, vira `dead` e você só assiste
- o resto: as `casas` da sua vizinhança, separadas por `;`

# Anatomia de uma casa

```
9,5,plain,0,0
```

- `9,5`: a posição - coluna 9, linha 5
- `plain`: o terreno
- `0`: o dono - `0` significa **casa vazia**; qualquer outro número é o dono
- `0`: fortificada ou não - `1` significa fortificada

a fortificação é **visível pra todo mundo**: dá pra ver o forte do inimigo antes de atacar. isso vira estratégia no exercício 3.

o contrato completo é o `PROTOCOL.md`, uma página, dentro do kit. em caso de dúvida, ele é a lei.

# Pergunta pra sala

Seu cliente manda `expand 12 7`. Um segundo depois, chega uma resposta do servidor.

**Como você sabe a qual pedido essa resposta se refere?**



# A etiqueta: o ref

- toda mensagem do cliente carrega um **ref**: uma etiqueta única, tipo `aurora-k3f9-17`
- o formato é `<nome>-<sessão>-<número>` : o número cresce a cada mensagem
- a **sessão** (4 letras, sorteada a cada vez que a página abre) garante que ninguém repete etiqueta, nem depois de reconectar
- quando o servidor responde `ACK aurora-k3f9-17`, ele **repete a etiqueta**: é assim que você casa resposta com pedido

# Pergunta pra sala

Você mandou `expand`. Passou o tick e **nenhuma resposta chegou**.

**O que aconteceu? E o que seu cliente deve fazer?**

# Tentar de novo - sem aplicar duas vezes

- o cliente não sabe se o pedido ou a resposta se perderam - então ele **reenvia**, com a **mesma etiqueta**
- o servidor guarda as etiquetas que já aplicou: se o ref é repetido, ele **não aplica de novo** - só reenvia
  - o **ACK** original
- reenviar é seguro: no máximo, você recebe a mesma confirmação duas vezes

é por isso que pagamento online não cobra duas vezes quando você clica "pagar" de novo: a etiqueta única é a trava.

# Quando a resposta é "não": o NACK

- nem toda resposta é `ACK`: se a jogada é inválida, o servidor responde `NACK`
- o `NACK` traz um **código de erro** de uma tabela fixa: `bad_cell`, `not_empty`, `not_enemy`, `too_late` ...
- o código é pra **máquina** ler (curto, fixo); o resto da linha é texto livre, pro **humano** ler
- `too_late`: sua ação chegou depois do tick fechar - ela entra na fila do próximo tick, **nunca some em silêncio**

erro aqui não quebra nada: é só mais uma mensagem, com informação pro seu cliente decidir o que fazer.

# Traduzir ou quebrar: o parser

- o que chega da rede é **texto cru**: uma linha que pode estar certa, errada ou pela metade
- antes de usar, o cliente **traduz**: a linha vira um dado organizado, com campos nomeados
- essa tradução se chama **parser** - e ela acontece **uma vez só, na entrada**
- depois dela, nenhum outro pedaço do código toca em texto cru

regra de ouro: linha malformada vira **resposta de erro**, nunca exceção. o programa não pode cair porque a rede mandou lixo.

# O leitor tolerante

- e se chegar uma linha com **campos a mais** no final?
- resposta: **ignore os extras** e fique com o que você conhece
- parser que exige o formato exato quebra na primeira mudança; parser tolerante sobrevive

guarde essa regra. às 2:50 o protocolo muda - e ela é a diferença entre consertar em 2 minutos ou reescrever tudo.

# PARTE 3



## O código

função pura, testes e o mapa do kit

# Pergunta pra sala

Como você testa um programa que depende de rede, servidor e outros 19 jogadores...

**sem rede, sem servidor e sem os outros 19?**



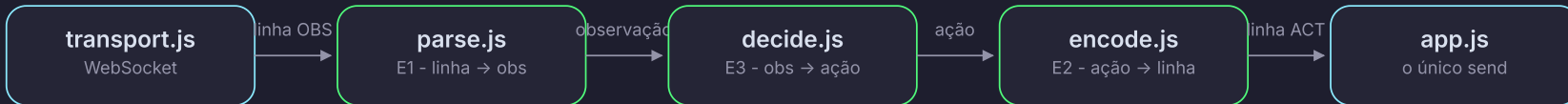
# Função pura: a receita de bolo

- uma **função pura** é como receita de bolo: mesmos ingredientes, mesmo bolo. sempre.
- ela não olha relógio, não abre rede, não sorteia número escondido: só usa o que entra por parâmetro
- mesma entrada, mesma saída - então dá pra testar **sem ligar nada**: entra uma linha de texto, sai um dado

a parte que fala com a rede (a **casca**) fica separada da parte que pensa (o **núcleo puro**). vocês só escrevem o núcleo.

# A arquitetura do cliente

CASCA - EFEITOS - JÁ VEM PRONTO / NÚCLEO PURO - VOCÊS ESCREVEM



**verde** = núcleo puro (vocês escrevem) / **azul** = casca (já vem pronta)

# No pipeline inteiro, só uma chamada fala com a rede.

---

encontrem ela quando investigarem o kit. todo o resto é função pura: mesma entrada, mesma saída, testável sem internet.

# O mapa do kit oficial (JavaScript)

```
student-kit/
├─ index.html          # abre no navegador, sem instalar nada
├─ src/
│   ├─ protocol/       # E1 parse.js - E2 encode.js
│   ├─ strategy/       # E3 decide.js - a sua estratégia
│   ├─ world/          # ajudantes prontos: expandable, attackable, borderCells
│   └─ infra/          # a casca: transporte, app, desenho, simulador local
├─ test/               # página de testes (começa toda vermelha)
└─ docs/               # PROTOCOL.md + cheatsheet
```

baixar em `/kit`, descompactar, `python3 -m http.server` na pasta, abrir a página no navegador, clicar em "testes". sem Node, sem build, sem internet.

# Por que o kit oficial é em JavaScript?

- todo computador do laboratório **já tem navegador**: nada pra instalar
- o kit JS é o único com **visualizador**: você vê a grade, as colônias e o placar na tela
- e é o único com **simulador local**: um servidor de mentira dentro da página, com dois bots pra treinar
- os testes rodam numa página: vermelho vira verde, sem terminal

a linguagem é só a ferramenta - os conceitos (protocolo, parser, função pura) são os mesmos em qualquer uma.

# Quer se aventurar em outra linguagem?

- cada linguagem tem seu próprio zip: `/kit/elixir`, `/kit/golang`, `/kit/python`, `/kit/c`
- todos falam **o mesmo protocolo** e jogam no **mesmo servidor** - muda a linguagem, não o jogo
- estrutura parecida em todos: um arquivo pro protocolo, um pra estratégia, um pros ajudantes, um pra casca

regra da casa: sem visualizador, sem simulador, **sem suporte do instrutor**. escolham pela linguagem que vocês dominam - em dúvida, fiquem no JS (`/kit`).

# Testes: do vermelho ao verde

- a página de testes começa **toda vermelha** - não é bug: é o mapa do que falta construir
- cada exercício que fica verde destrava a próxima fase
- o **simulador local** tem exatamente a mesma lógica do servidor real - o que passa nele passa no torneio

meta do próximo bloco: E1 e E2 verdes, e a sua colônia aparecendo no placar do simulador.

# PARTE 4



## Mão na massa

E1 a E3, um conceito por vez



# E1 – o tradutor da entrada

parse.js: a linha de texto vira um dado organizado

```
// src/protocol/parse.js
parseObservation("OBS srv-97 97 alive 97 9,5,plain,0,0;...")
// ⇒ { tag: "ok",
//      value: { ref, tick: 97, status: "alive", scoresTick, cells } }

parseObservation("OBS srv-97 alive") // linha quebrada
// ⇒ { tag: "error", reason: "..." } // nunca exceção
```

- a função **devolve** um resultado com etiqueta: **ok** com o valor, ou **error** com o motivo – quem chamou decide o que fazer
- **parseCell** e **cellList** já existem no kit: o trabalho é montar a linha inteira
- campos extras no final da linha: **ignore** – é o leitor tolerante do slide anterior

## E2 - o tradutor da saída

encode.js: o espelho do E1 - a ação vira linha de texto

```
// src/protocol/encode.js
encodeAction({ kind: "expand", x: 12, y: 7 }, ref)
// ⇒ "ACT aurora-k3f9-17 expand 12 7"

encodeAction({ kind: "pass" }, ref)
// ⇒ "ACT aurora-k3f9-18 pass"           // sem coordenadas
```

- o ref vem pronto de `createRefs(nome)`: `<nome>-<sessão>-<número>`, número crescente
- lembra da etiqueta? é **essa** função que coloca ela na mensagem - e é ela que permite reenviar sem medo

# E3 - o cérebro

decide.js: a única função que é 100% sua - olha a vizinhança, escolhe a ação

- pura como as outras: mesma observação, mesma ação - sem rede, sem relógio, sem sorte escondida
- ajudantes prontos em `world/observation.js`: `expandable`, `attackable`, `borderCells`
- **começo simples**: expanda pra uma casa vazia vizinha; nunca ataque casa fortificada
- **próximo nível**: fortifique fronteira com inimigo ao lado; ataque inimigo sem fortificação
- os testes só verificam o **tipo** da ação, não a casa exata: a estratégia é livre

meta: E3 verde + uma partida completa contra os bots do simulador.

# Quando as regras mudam: a v2

- às 2:50 o servidor reinicia na **v2**: cada casa ganha um campo a mais (recurso)
- protocolo novo não renomeia nem reordena campo: evolução é **adicionar no final**
- parser que exige exatamente 5 campos **quebra**; parser tolerante sobrevive
- todo mundo quebra do mesmo jeito, todo mundo conserta junto, em 10 minutos

é pra isso que existe o **v1** no **HELLO** - e o leitor tolerante do E1.

# PARTE 5



## O torneio

e o que acontece do outro lado, no servidor

# Duas fases, um restart

- **cooperativa** (2:30): servidor com ataques desligados; `attack` recebe `NACK attacks_disabled`
- cada colônia cresce no seu canto - agora contra o servidor **de verdade**, na rede **de verdade**
- **torneio** (3:00): o servidor reinicia sem a trava - restart é partida nova, de propósito
- rodadas curtas; quem perde assiste pelo projetor

se a conexão cair: reconectar com o mesmo nome de colônia viva retoma o estado. colônia morta é entrada nova.

# A demo final



- o servidor usa **exatamente** a arquitetura que vocês acabaram de escrever: núcleo puro no meio, rede na borda
- a trava de etiqueta repetida cabe em ~5 linhas de Elixir - mostrada ao vivo
- o servidor grava cada evento num log: reexecutar o log dá o mesmo placar - o estado é só a soma dos eventos
- e sim: vou derrubar o servidor no meio de uma partida. observem os `NACK too_late` chegando

# O que fica

---

O jogo era a desculpa. O que vocês praticaram vale pra qualquer sistema:

um **contrato** na fronteira, um **núcleo puro** no meio, os **efeitos** na borda.

E isso funciona em qualquer linguagem.



# Obrigada

em todo lugar: @zoedsoupe

zoedsoupe.zeetech.io - slides + referências



## Bom torneio.